# AttentionLens: A Toolkit for Comparing Attention and Token-Mixing Patterns Across GPT-2, BERT, RetNet and RWKV

Mitra Varun Gogulapati
*Independent Researcher*
mitravarun2702@gmail.com

***Abstract*—Researchers who want to compare how different sequence models route information between tokens run into a practical problem. GPT-2 and BERT expose softmax attention matrices directly. RetNet computes signed retention scores, and RWKV never forms an attention matrix at all. Comparing them means writing separate extraction code for each model, and the resulting numbers are rarely computed the same way. We present AttentionLens, a toolkit that extracts a native attention or token-mixing analogue from GPT-2, BERT, a TorchScale RetNet and a pretrained RWKV-4 model, and stores all of them in one tensor format with layers, heads, query positions and key positions. On top of that format the toolkit computes descriptive metrics (normalised entropy, relative attention distance, first-token mass and lookback offset profiles), applies a rule-based head classifier, projects heads with PCA, and serves token-linked heatmaps through a web interface. We apply it to 32 hand-written English prompts, which gives 576 head or head-group instances across four checkpoints. In this setting GPT-2 sends most of its attention to the first token (62.6% of mass on average, and 106 of 144 heads classified as first-token heads). BERT shows little first-token mass (12.4%) and most of its heads fall outside our simple rules. The RetNet checkpoint, which we trained briefly on TinyStories, has the highest normalised entropy (0.860) and the shortest relative distance (23.0%). RWKV-4 sits between the others on most measures. A threshold sensitivity analysis shows that head-type counts change substantially when the classification thresholds move by 0.1. We treat all findings as descriptions of these specific checkpoints on this prompt set. The study does not isolate architectural effects and makes no causal claims about model predictions.**

***Index Terms*—attention analysis, interpretability, retentive networks, RWKV, attention sinks, visualisation toolkit**

## I. Introduction

Attention weights are one of the most inspected objects in modern language modelling [1], [2]. A single heatmap can show that a head follows the previous token, fixates on a delimiter, or spreads its mass over the whole sentence, and that is often enough to start a useful investigation. Most of this work was done on softmax Transformers such as BERT [3] and GPT-2 [4], and several well-known tools exist for browsing their attention [5], [6].

The set of sequence models that people actually run is now wider. RetNet replaces softmax attention with a decayed retention operator that has a parallel form, a recurrent form and a chunked form [7]. RWKV reformulates a Transformer-like model as a linear recurrence over channel-wise decayed key-value sums [8]. These models are built so that they can be computed without a full attention matrix at inference time. For the same reason, they do not hand an analyst the object that attention tooling expects.

Recent work on implicit attention has shown that such models can still be written as data-controlled mixing matrices over tokens [9], [10]. Once you do that, the same questions can be asked of every model. How concentrated is the mixing? How far apart are the tokens it connects? Does a large share of it go to the first position? How many heads follow a simple positional pattern? The difficulty is that each model needs its own extraction path, each path has its own normalisation conventions, and small differences in those conventions can change the headline numbers.

AttentionLens is our attempt to remove that overhead. It does not propose a new interpretability method. It standardises extraction and measurement, so that differences in a plot come from the models and not from how each one was processed. Our contributions are as follows.

1) A common tensor interface (layers by heads by query by key, plus tokens and flags for causality and nativeness) with extractors for GPT-2, BERT, TorchScale RetNet and RWKV-4. For RWKV we rebuild per-token weights from the WKV recurrence and average channels into pseudo-heads.
2) A fixed set of descriptive metrics and a documented rule-based head classifier. The sign handling for RetNet, the exact rollout procedure and the treatment of BERT's special tokens are written down explicitly.
3) A small comparative study with 32 shared prompts, 576 head instances and six paper figures, together with a threshold-sensitivity analysis that shows which conclusions are fragile.
4) A local web application (FastAPI backend, React frontend) with token-linked heatmaps and an exact-prompt cache, and a script pipeline that regenerates every figure and table in this paper from saved tensors.

We want to be clear about what the study is. The four checkpoints differ in training data, objective, tokenisation, size and training maturity, and our RetNet is a short TinyStories run, not a matched pretrained model. The results therefore describe four particular checkpoints. They do not rank architectures.

The remainder of the paper is organised as follows. Section II reviews related work. Section III describes the system.

Section IV gives the methodology and Section V the experimental setup. Results are in Section VI, followed by discussion (Section VII), limitations (Section VIII) and conclusions (Section IX).

## II. Related Work

Attention analysis in Transformers. Soon after BERT appeared, several studies catalogued what its heads do. Clark et al. [11] found heads that attend to delimiters, to the next or previous token, and to syntactic relations such as direct objects. Kovaleva et al. [12] reported that many heads in fine-tuned BERT follow a small number of repeated patterns, and that disabling many of them barely changes task performance. Voita et al. [13] showed that in a translation model a small group of heads with identifiable roles did most of the work and the rest could be pruned, while Michel et al. [14] reached a related conclusion for BERT and machine translation by testing head removal directly. Tenney et al. [15] took a different route and probed layer representations, finding that they follow a pipeline resembling classical NLP stages. Our classifier is in the spirit of the positional categories in this literature, but it is deliberately simple and makes no claim about head function.

Attention sinks. Xiao et al. [16] observed that autoregressive models place a large amount of attention on the first few tokens regardless of their semantic content, and used this to build a streaming inference scheme. First-token mass is one of our main metrics for this reason. We treat it as a descriptive quantity and make no claim about why a given checkpoint shows or lacks a sink.

Visualisation tools. BertViz [5] provides head, model and neuron views of attention for Transformer models, and exBERT [6] adds search over annotated corpora and aggregated attention across a dataset. Both are built around softmax attention in Transformers. AttentionLens differs mainly in scope. It targets cross-architecture comparison under one tensor format and ships the metric code used to produce the figures.

Cumulative attention and rollout. Raw attention at a single layer says little about how information moves through the network, since residual connections and layer mixing intervene. Abnar and Zuidema [17] proposed attention rollout and attention flow, which combine layer attention matrices (with residual connections added) to approximate token-to-token influence at the output. Our rollout follows the rollout recipe with the specific choices described in Section IV-D. We use it as a visual summary only.

Attention and explanation. Whether attention weights explain predictions has been disputed. Jain and Wallace [18] showed that attention weights often correlate weakly with gradient-based importance and that very different attention distributions can give the same output. Wiegreffe and Pinter [19] argued that the answer depends on how the question is posed and that attention can still offer plausible explanations under some conditions. Serrano and Smith [20] ran intermediate-representation erasure tests and found that attention singles out important representations imperfectly. Kobayashi et al. [21] pointed out that attention weights ignore the norm of the value vectors they scale, so the weight alone understates or overstates a token's contribution. Bastings and Filippova [22] argued that saliency methods are a better fit when the goal is explanation. These results shape how we write about our own numbers. A head with high entropy, or one that sends 60% of its mass to the first token, is described as such and nothing more.

Retention and recurrent token mixing. RetNet [7] defines retention with an exponential decay between positions. It has a parallel form for training, a recurrent form for inference and a chunkwise form that sits between them. We use the TorchScale implementation [23]. RWKV [8] is a linear-attention-style recurrent model whose time-mixing block accumulates key-value pairs with a learned per-channel decay. The idea of casting linear attention as a recurrence is older [24], and the broader family of selective state space models includes Mamba [25].

Implicit attention. Ali et al. [9] showed that Mamba layers can be expressed as data-dependent attention-like matrices and used them for visualisation and explainability. Zimerman et al. [10] extended this to a unified formulation covering several gated linear recurrent models, including RWKV. We use the same general idea of reading off a token-mixing matrix, but our RWKV reconstruction is a lighter one. It uses only the WKV weights and leaves out token shift, the receptance gate and the output projection, and Section VIII discusses what that omission costs.

Mechanistic work on heads. Elhage et al. [26] and Olsson et al. [27] studied circuits and induction heads in small Transformers using intervention and composition analyses. We do not perform interventions. Our classifier cannot detect induction behaviour, and we flag this wherever the labels might be over-read.

## III. System Architecture

### A. Overview

The pipeline has four stages. In the first, prompts are read from a YAML file and tokenised with each model's own tokenizer. The second stage runs the extractor registered for that model. The third stage writes each result as a saved prompt record. The fourth stage reads saved records, computes metrics, and produces either static figures or interactive views.

### B. Extractor registry

Extractors are registered lazily, so importing the package does not load model weights. Each extractor wraps one model family and exposes a single method that takes text and returns a result object. GPT-2 and BERT are run with eager attention so that softmax weights are returned. For RetNet we hook the parallel retention computation. For RWKV-4 we hook the key tensors and rebuild the WKV weights (Section IV-B). Because the hooks mutate model state, extraction on a shared model instance is guarded by a lock.

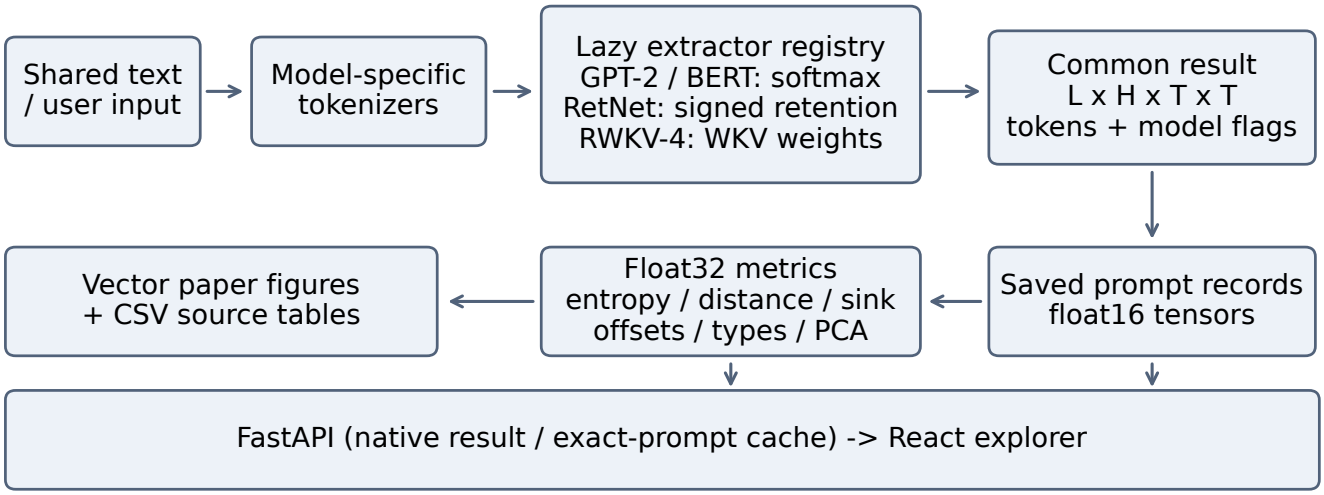



Fig. 1. AttentionLens pipeline, redrawn from the inspected extractor, saved-record, metric and web-API code. Shared strings use model-specific tokenizers; saved tensors are float16 and metric calculations use float32.

### C. Common result format

Every extractor returns a tensor of shape $L \times H \times T \times T$, where L is the number of layers, H the number of heads or head groups and T the number of tokens. The result also carries the token strings and flags that say whether the model is causal and whether the matrices are native attention or an analogue. For the four models in this study $L = 12$ and $H = 12$, giving 144 head or head-group instances per model.

### D. Saved-data workflow

Extraction runs on CPU in evaluation mode without gradient tracking. Saved tensors are stored in float16 to keep the records small, and every metric casts them back to float32 before computing. This is a convenient place for small numeric differences to creep in, so we state it as part of the protocol (Section V-E). The scripts that produce results are run in a fixed order: extraction, metrics, figures, supplementary analysis figures, sensitivity analysis and the paper-figure script. The full list of commands is in the Code and Data Availability section.

### E. Python API

The public extractor factory is `attentionlens.extractors.build(name, cfg)`. Its returned object exposes `extract(text, max_len=128)` and returns an `AttentionResult` containing `attn`, `tokens`, `model_name`, `causal` and `native`. The following calls use paths relative to the repository root.

```
from attentionlens.extractors import build
from attentionlens.utils.io import load_yaml
cfg = load_yaml("configs/models.yaml")
ext = build("gpt2", cfg["gpt2"])
result = ext.extract(
   "The quick brown fox jumps over the lazy dog."
)
```

Saved files are lists of dictionaries, one per prompt, with `text`, `tokens` and `attn`. Flags are supplied from the model configuration when loading records; they are not fields in the saved dictionary.

```
from attentionlens.utils.io import load_tensors
from attentionlens.metrics.entropy import head_entropy
from attentionlens.metrics.head_types import head_scores
from attentionlens.metrics.rollout import attention_rollout
rec = load_tensors("outputs/tensors/gpt2.pt")[0]
a = rec["attn"].float()
entropy = head_entropy(a, causal=True)
scores = head_scores(a, window=3)
rollout = attention_rollout(a)
```

The low-level functions operate on one prompt tensor. `scripts/compute_metrics.py` aggregates the head metrics and applies `classify`; `scripts/make_paper_figures.py` adds relative distance and offset summaries and exports the numerical tables.

### F. Web application

A FastAPI service exposes three groups of endpoints. The model listing endpoint reports which models are available, and RetNet availability depends on whether its checkpoint file is present. The extraction endpoint returns attention matrices, cumulative rollout, entropy, tokens and model flags. If the submitted text exactly matches a saved prompt, stored tensors are reused. Otherwise the model is run. The analysis endpoints return dataset metadata and the interactive figures. Model instances and analysis data are cached. The React frontend is served from the same origin as the API, and both run locally on port 8501. The interface shows token-linked heatmaps, so that selecting a token highlights its row or column across views, and a head-comparison grid.

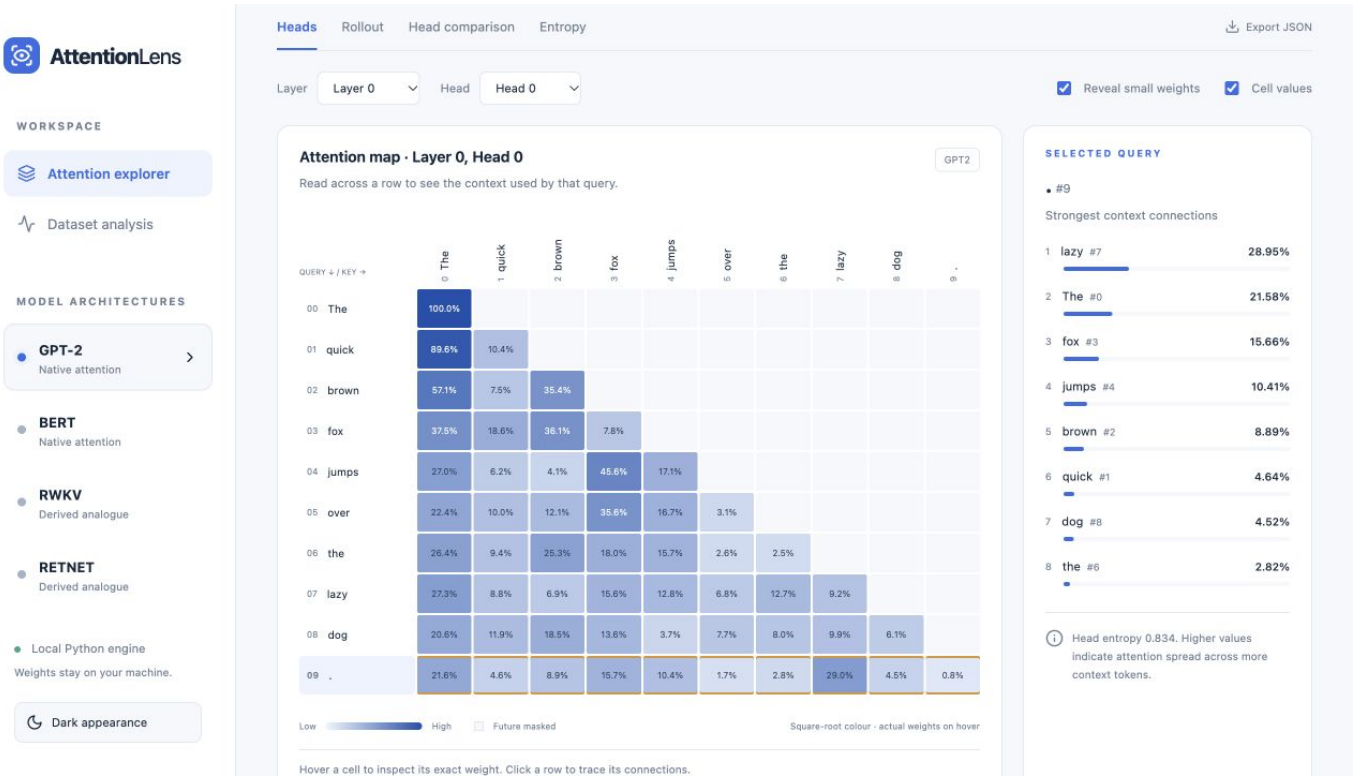



Fig. 2. Actual React explorer on the first saved GPT-2 prompt, layer 0, head 0. The selected period query is highlighted across the heatmap and strongest-connection panel. Cell values are percentages; the screenshot uses square-root colour contrast.

## IV. Methodology

### A. Notation

For a model with L layers and H heads, let $A^{(l,h)} \in \mathbb{R}^{T\times T}$ denote the extracted matrix for layer l and head h, with entry $A_{ij}$ giving the weight from query position i to key position j. Positions are indexed from zero. For causal models, $A_{ij} = 0$ for $j > i$.

### B. Extraction per model

GPT-2 and BERT. We take the softmax attention weights from every layer and head. GPT-2 is causal. BERT is bidirectional, so each row distributes over all T tokens, including [CLS] and [SEP].

RetNet. TorchScale's parallel retention computes a score matrix from queries and keys and multiplies it elementwise by a decay mask before applying it to the values. In its simplest form,

$$\begin{aligned}\mathrm{Retention}(X) &= (QK^\top \odot D)V,\\ D_{nm} &= \begin{cases}\gamma^{n-m} & n \geq m,\\ 0 & n < m.\end{cases}\end{aligned} \tag{1}$$

where $\gamma$ is a per-head decay rate. We hook the matrix applied to V.

Inspection of the supplied extractor and TorchScale 0.3.0 confirms that the hook receives rotated queries and keys, with keys already scaled by $d_k^{-1/2}$. Its mask is the causal decay matrix divided row-wise by the square root of its row sum. If $C = (Q_r K_r^\top) \odot \widetilde{D}$, the stored matrix is

$$A_{ij} = \frac{C_{ij}}{\max\left(\left|\sum_k C_{ik}\right|, 1\right)}. \tag{2}$$

The denominator is detached from autograd in the implementation. The hook therefore includes both decay-mask normalisation and signed-row score scaling, and records the matrix multiplied by the projected values before group normalisation, gating and output projection.

These weights are signed and rows are not constrained to sum to one. For all metrics we work with absolute values, which discards sign and any cancellation that the real computation relies on. This is a limitation, discussed in Section VIII.

RWKV-4. RWKV-4 has no attention matrix. Its time-mixing block computes, for each channel c and position t, a weighted average of value vectors,

$$\begin{aligned}z_{t,i,c} &= \begin{cases}e^{-(t-1-i)w_c + k_{i,c}} & i < t,\\ e^{u_c + k_{t,c}} & i = t,\end{cases}\\ \mathrm{wkv}_{t,c} &= \frac{\sum_{i=0}^{t} z_{t,i,c} v_{i,c}}{\sum_{i=0}^{t} z_{t,i,c}}.\end{aligned} \tag{3}$$

where $w_c \geq 0$ is the learned channel decay, $u_c$ is a learned bonus for the current token and k are the keys. The coefficient multiplying $v_{i,c}$ is a weight over earlier tokens that sums to one across $i \leq t$ for each channel. We hook the key tensors, read w

and u from the checkpoint, and compute these weights directly. Writing $a_{t,i,c}$ for the normalised coefficient on position i, we form 12 pseudo-heads by averaging 64 contiguous channels,

$$A_{ti}^{(l,g)} = \frac{1}{64} \sum_{c \in G_g} a_{t,i,c}^{(l)}, \qquad G_g = \{64g, \ldots, 64g + 63\}. \tag{4}$$

for $g = 0, \ldots, 11$. Rows still sum to one. The grouping is an analysis choice that we introduced. RWKV has no native heads. The reconstructed weights leave out token shift, the receptance gate and the output projection, so they describe only part of what the time-mixing block does.

### C. Normalisation

Because RetNet values are signed, and because we want one definition for all models, every head metric starts from absolute values renormalised by row:

$$p_{ij} = \frac{|A_{ij}|}{\sum_{j'} |A_{ij'}|} \tag{5}$$

For GPT-2, BERT and RWKV this leaves the matrix unchanged. For RetNet it produces a probability-like distribution that no longer carries sign.

### D. Cumulative rollout

Rollout is used for Fig. 3 and for the explorer. Our procedure follows Abnar and Zuidema [17] but is not identical to it. For each layer l we average absolute weights over heads without normalising each head first,

$$M_l = \frac{1}{H} \sum_h |A^{(l,h)}| \tag{6}$$

add the identity to account for the residual connection and renormalise rows,

$$B_l = \mathrm{RowNorm}(M_l + I) \tag{7}$$

and compose layers from the input upward,

$$R_l = B_l R_{l-1}, \qquad R_{-1} = I \tag{8}$$

Because heads are not individually normalised before averaging, the relative weight of the identity term depends on the scale of the input matrices. For softmax models, rows sum to one and the identity receives a fixed share. For RetNet, whose rows are signed and unnormalised, the effective weight of the residual term depends on input scale, so we do not describe the procedure as a fixed 50/50 mixture of attention and residual. Rollout here is a descriptive proxy for how mass spreads across layers.

### E. Metrics

All metrics below are computed per head and per prompt, then averaged across prompts. T denotes the token count for that prompt.

Normalised entropy. For query i with $n_i$ admissible keys ($n_i = i + 1$ for causal models, $n_i = T$ for BERT),

$$H_i = -\frac{1}{\log n_i} \sum_j p_{ij} \log p_{ij} \tag{9}$$

The head value is the mean over queries. For causal models query zero is omitted, since $n_0 = 1$ and its entropy is zero by construction. A value of 1 means uniform attention over the admissible keys, and a value near 0 means the row is concentrated on one token. A one-token prompt returns zero.

Relative distance.

$$D = \frac{\sum_{i,j} p_{ij} |i - j|}{T(T-1)} \tag{10}$$

This measures the average token separation relative to the largest possible one. The normalisation is by token position, not by word.

First-token mass.

$$S = \frac{1}{T-1} \sum_{i=1}^{T-1} p_{i0} \tag{11}$$

For BERT, position zero is the [CLS] token, not the first token of the text, so S measures attention to [CLS] there.

Lookback offset. For an offset $d \geq 0$,

$$O_d = \frac{1}{|V_d|} \sum_{i \in V_d} p_{i,i-d}, \qquad V_d = \{i : i \geq d\}. \tag{12}$$

Values are averaged over heads and layers within a prompt, and then averaged over prompts that have at least one valid query at that offset. Offset zero is self-attention. Larger offsets have fewer eligible prompts, which we report alongside the curves. Forward-looking BERT weights are excluded from this measure.

### F. Reference values for uniform attention

Raw numbers are hard to read without a baseline, so we derived the values each metric takes when every row is uniform over its admissible keys. These come from the definitions above and are included for orientation only.

Entropy equals 1 by construction. For a causal model with uniform rows, the relative distance is exactly 1/4, since the mean distance in row i is $i/2$ and the sum over rows is $T(T-1)/4$. For bidirectional uniform rows it is $(T+1)/(3T)$, which is about 0.36 for $T = 11$ and about 0.35 for $T = 25$. First-token mass under uniform causal rows is $(H_T - 1)/(T - 1)$, where $H_T$ is the T-th harmonic number. For the saved token range of 9 to 23 this falls between roughly 23% and 12%. For BERT with uniform rows it is $1/T$, between about 9% and 4%.

TABLE I
CLASSIFICATION PRIORITY FROM THE SUPPLIED CONFIGURATION AND CODE; THE FIRST PASSING RULE WINS.

| Priority | Type | Criterion |
|---|---|---|
| 1 | First token | Mean $p_{i0} \geq 0.5$ |
| 2 | Previous token | Mean $p_{i,i-1} \geq 0.5$ |
| 3 | Next token | Mean $p_{i,i+1} \geq 0.5$ |
| 4 | Self | Mean $p_{ii} \geq 0.5$ |
| 5 | Local window | Window $\pm 3$: mass $\geq 0.7$ |
| 6 | Broad | Entropy $\geq 0.8$ |
| 7 | Other | No earlier rule passes |

We verified these baselines against `head_entropy`, `head_scores` and the figure code's `_mean_distance` using uniform matrices for $T = 2, \ldots, 25$. For $T = 1$, entropy, distance and first-token mass return zero. The implementation divides absolute rows by their sum clamped below at $10^{-9}$; a zero row stays zero, rather than becoming a uniform distribution. Entropy uses $\log(\max(p, 10^{-12}))$.

### G. Head classification

Each head receives one label from a fixed priority list, using prompt-averaged scores. The first rule that passes wins.

The first-token test uses the mean of $p_{i0}$ over $i \geq 1$. The previous-token, next-token and self tests use the mean of $p_{i,i-1}$, $p_{i,i+1}$ and $p_{ii}$ over valid queries, with a threshold of 0.5. The local-window test sums row mass over $|i-j| \leq 3$ and requires 0.7. A head that matches nothing earlier is labelled broad if its normalised entropy is at least 0.8 and other otherwise. These labels describe simple positional patterns. They do not show what a head is for, and the classifier cannot detect induction, syntactic or delimiter behaviour.

### H. PCA head atlas

To see how heads group, we build an 11-dimensional feature vector for every head from prompt-averaged values: offsets 0 to 5, mass on later positions (ahead mass), normalised entropy, first-token mass, local-window mass and relative distance. Features are standardised across all heads from all models, and we project with singular value decomposition onto the first two components. This atlas is separate from the explorer's head-comparison grid, which shows individual heads side by side without projection.

### I. Aggregation

Each prompt contributes equally to an aggregate mean. We report population standard deviations over the 32 prompts. These describe how much the metric varies from prompt to prompt, and they are not confidence intervals for a population mean.

## V. EXPERIMENTAL SETUP

### A. Models

GPT-2 is the 12-layer, 12-head base model from the Hugging Face hub (gpt2) [4], [28]. BERT is bert-base-uncased [3]. RWKV-4 is the checkpoint `RWKV/rwkv-4-169m-pile`, trained on the Pile [8], [29]. RetNet is a TorchScale model [23] with 12 layers, 12 heads, embedding and value width 768 and feed-forward width 3072, loaded from `checkpoints/retnet_128.pt`.

The RWKV checkpoint name refers to the full language model. AttentionLens loads its backbone without the output head. The full causal language-model class has 169,342,464 parameters and the extracted backbone has 130,729,728. The 12 pseudo-heads are the grouping from Section IV-B and are not a native feature of RWKV.

The parameter counts differ widely. The RetNet model has roughly twice as many parameters as GPT-2 or RWKV in the extractor class, partly because of its feed-forward width. We do not match parameter counts and report them so readers can account for the difference.

### B. RetNet training

No public pretrained RetNet checkpoint was used. We trained one briefly on TinyStories [30] with the GPT-2 tokenizer. The author confirms 1,000 optimiser steps, sequence length 128, batch size 1, gradient accumulation 8 and 100 validation batches. These settings imply a nominal $1{,}000 \times 128 \times 1 \times 8 = 1{,}024{,}000$ processed target tokens if every step completed all microbatches; they do not count unique tokens.

The supplied notebook uses AdamW with peak learning rate $3 \times 10^{-4}$, weight decay 0.1 and betas $(0.9, 0.95)$, seed 42, gradient clipping at 1.0, and float16 autocast with gradient scaling on a Tesla T4. Its schedule is linear warmup followed by cosine decay to 10% of the peak rate. For the author-confirmed 1,000-step setting, the written schedule gives 100 warmup steps.

There is an unresolved provenance discrepancy. The supplied `training_metadata.json` and saved training output record 100 steps, final training loss 4.5150 and validation loss 4.4345. Saved output also prints 10 validation batches, whereas the notebook's current configuration and the author's correction specify 100. Notebook execution counters are not in source order. We retain the recorded losses as properties of the supplied 100-step record; they are not verified losses of a 1,000-step run. The correspondence between the author-confirmed training run, the exported checkpoint and the saved RetNet analysis tensors needs a matching execution record. Appendix D separates these sources explicitly.

Checkpoint inspection and the notebook's model summary give a value width of 768 and 236,185,681 parameters. Both 100 and 1,000 steps are brief training regimes relative to mature pretrained checkpoints; the RetNet results should therefore be read as descriptive measurements of the supplied saved tensors, with training provenance qualified as above.

### C. Prompts

The prompt set consists of 32 English strings written by hand and stored in configs/prompts.yaml. Every model receives exactly the same strings.

Inspection of the strings shows a mixture of everyday narratives, statements about geography and machine learning, questions, imperatives, conditionals, relative clauses and

TABLE II
MODEL CONFIGURATIONS USED BY THE EXTRACTORS. RWKV GROUPS ARE AN ANALYSIS CONVENTION.

| Model | Source | Mechanism | Direction | Dimensions |
|---|---|---|---|---|
| GPT-2 | `gpt2` | Softmax attention | Causal | 12 layers, 12 heads; width 768 |
| BERT | `bert-base-uncased` | Softmax attention | Bidirectional | 12 layers, 12 heads; width 768 |
| RetNet | `checkpoints/retnet_128.pt` | Signed retention | Causal | 12 layers, 12 heads; embedding/value 768; FFN 3072 |
| RWKV-4 | `RWKV/rwkv-4-169m-pile` | WKV analogue | Causal | 12 layers; 768 channels; 12 groups of 64 |

TABLE III
UNIQUE PARAMETERS IN THE INSTANTIATED EXTRACTOR CLASSES AND SAVED TOKEN RANGES.

| Model | Parameters | Tokens |
|---|---|---|
| GPT-2 | 124,439,808 | 9–23 |
| BERT | 109,482,240 | 11–25 |
| RetNet | 236,185,681 | 9–23 |
| RWKV-4 backbone | 130,729,728 | 9–24 |

TABLE IV
OVERALL METRICS: MEAN ± POPULATION SD ACROSS 32 PROMPTS AFTER AVERAGING LAYERS AND HEADS WITHIN EACH PROMPT. DISTANCE AND FIRST-TOKEN MASS ARE PERCENTAGES; THEIR SDS ARE PERCENTAGE POINTS.

| Model | Entropy | Distance | First token |
|---|---|---|---|
| GPT-2 | $0.4820 \pm 0.0238$ | $35.44 \pm 1.12$ | $62.61 \pm 3.02$ |
| BERT | $0.5335 \pm 0.0188$ | $37.88 \pm 1.02$ | $12.42 \pm 1.09$ |
| RetNet | $0.8596 \pm 0.0130$ | $23.01 \pm 0.56$ | $16.00 \pm 2.91$ |
| RWKV-4 | $0.7451 \pm 0.0209$ | $25.84 \pm 1.49$ | $33.44 \pm 3.62$ |

quoted dialogue. They include names (Alice and Bob), a year, enumerated number words, punctuation, repeated words and potentially ambiguous pronouns. These are descriptive properties of the supplied list, not balanced experimental categories; no sampling or category-balancing procedure is recorded. The complete strings and measured token counts appear in Appendix A.

Models tokenise independently, so the same string produces different token counts: 9 to 23 for GPT-2 and RetNet, 11 to 25 for BERT (which adds [CLS] and [SEP]) and 9 to 24 for RWKV. Maximum extraction length was set to 128 tokens, and every saved prompt is far below that limit.

### D. Software and hardware

The supplied local environment inspected on 24 October 2026 contains Python 3.10.9, PyTorch 2.11.0 [31], Transformers 4.57.6 [28], TorchScale 0.3.0, NumPy 2.2.6, Matplotlib 3.10.9 [32], Plotly 7.1.0, FastAPI 0.142.2 and Uvicorn 0.54.0. The frontend lockfile resolves React 19.3.0 and Vite 6.4.3. This is an inspection of the supplied environment, not a contemporaneous record of the original extraction or Colab training run. Exact historical package versions, extraction OS and CPU model remain unrecorded. All extraction ran on CPU in evaluation mode without gradient tracking.

### E. Precision and aggregation

Tensors are saved in float16 and metrics are computed in float32. Each metric is first averaged across layers and heads within a prompt, then across the 32 prompts, with population standard deviation across prompts. Distance and first-token mass are reported as percentages, and their standard deviations are in percentage points. Layer-wise bands in Figs. 4 and 7 use per-layer prompt statistics.

## VI. RESULTS

All results below describe these four checkpoints on these 32 prompts.

### A. Overall metrics

Table IV gives the main numbers. GPT-2 has the lowest normalised entropy ($0.482 \pm 0.024$) and the largest first-token mass ($62.61 \pm 3.02\%$). BERT's entropy is slightly higher ($0.534 \pm 0.019$) and its first-token mass is much lower ($12.42 \pm 1.09\%$). The RetNet checkpoint has the highest entropy ($0.860 \pm 0.013$), the shortest relative distance ($23.01 \pm 0.56\%$) and a first-token mass of $16.00 \pm 2.91\%$. RWKV-4 is in between on entropy ($0.745 \pm 0.021$) and distance ($25.84 \pm 1.49\%$), with first-token mass of $33.44 \pm 3.62\%$.

Reading these against the uniform references from Section IV-F helps. RetNet's distance of 23.0% is slightly below the 25% expected for uniform causal rows, and its entropy is close to 0.86, so its averaged rows look like fairly spread-out distributions tilted toward recent tokens. Its first-token mass of 16.0% falls inside the 12 to 23% range that uniform causal rows would give at these lengths. RWKV's distance of 25.8% is near the uniform value. GPT-2's relative distance is 35.4%, which is well above the causal uniform value of 25%. This high figure comes mostly from the sink and does not indicate long-range content mixing, because mass placed on position zero is at distance i from the query. BERT's 37.9% is a little above the bidirectional uniform value of about 35 to 36%.

The standard deviations are small compared with the gaps between models for entropy and for first-token mass, which tells us the model differences are consistent across prompts. They do not tell us that the differences would hold on other prompt sets.

### B. Rollout on a shared sentence

The shared sentence is "The quick brown fox jumps over the lazy dog." The highlighted query is the sentence-final period: index 9 in the causal models and index 10 in BERT, before [SEP]. In that row, GPT-2 allocates 98.9% of rollout mass to the first token, compared with 93.3% for RWKV-4 and 26.4% for RetNet. BERT allocates 40.9% to [CLS] and 15.5% to

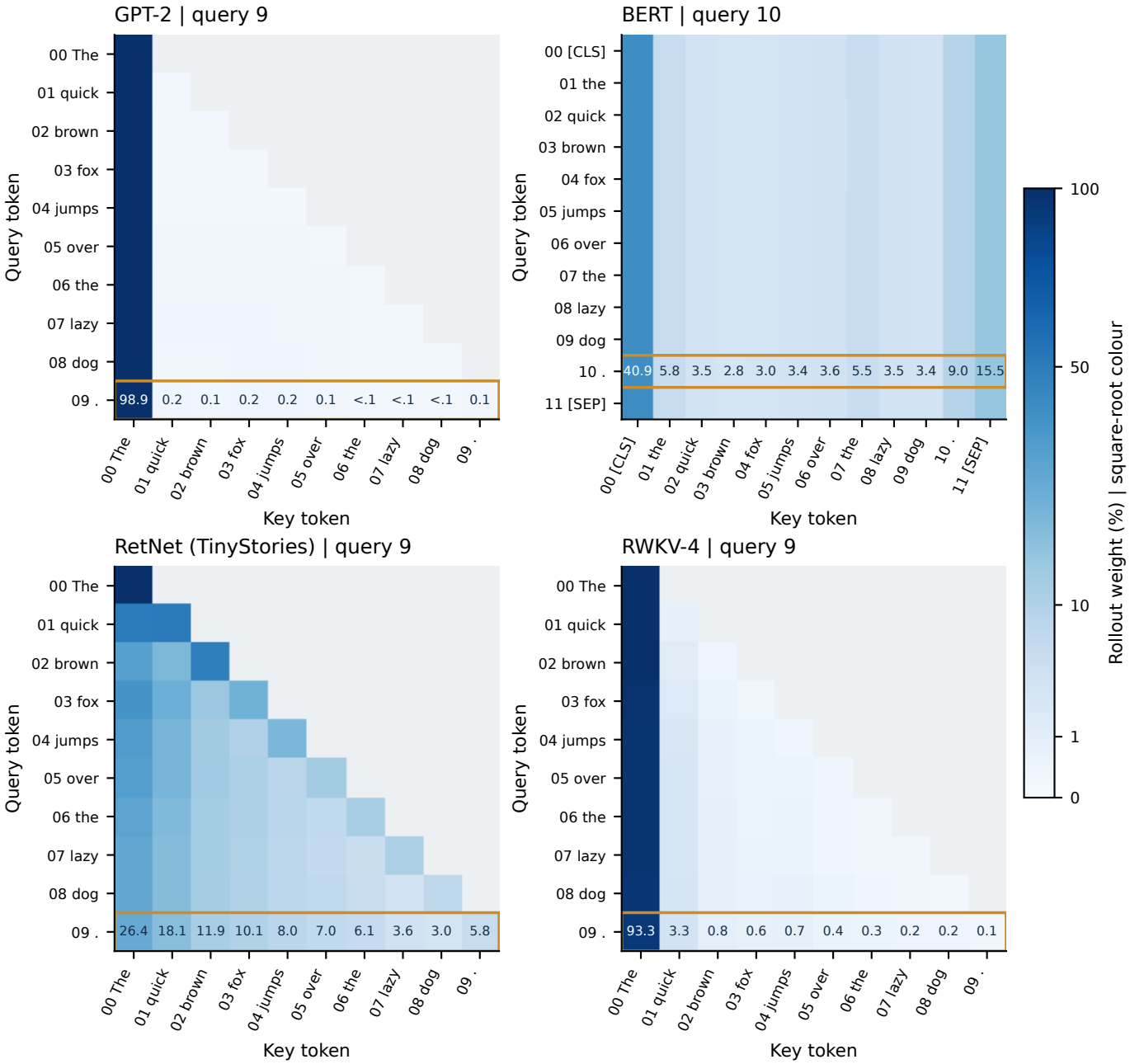



Fig. 3. Final-layer cumulative rollout for "The quick brown fox jumps over the lazy dog." The outlined row is the sentence-final period (before BERT [SEP]); annotations are percentages. All panels share a square-root colour mapping; grey denotes the causal mask. This is a single-input illustration, not a dataset average.

[SEP]; these are cumulative-rollout values for this sentence, not the raw final-layer first-token metric.

The figure illustrates, on one input, what the aggregate numbers suggest. Where GPT-2's rollout concentrates in the first column, the RetNet checkpoint's rows are more evenly spread. Because rollout depends on the residual treatment (Section IV-D), the RetNet panel should be read as a descriptive map and not as a measure of influence.

### C. Layer-wise entropy

At the final layer, RetNet's entropy is $0.937 \pm 0.012$, which is close to uniform, and RWKV's is $0.810 \pm 0.040$. GPT-2 stays at $0.473 \pm 0.050$ and BERT at $0.553 \pm 0.067$. For BERT and GPT-2 the final-layer values are close to their respective all-layer averages, but this does not imply flat layer-wise curves. For RetNet the final layer is noticeably higher than the all-layer mean of 0.860, and for RWKV it is higher than 0.745.

The CSV curves show a gradual RetNet increase from 0.819

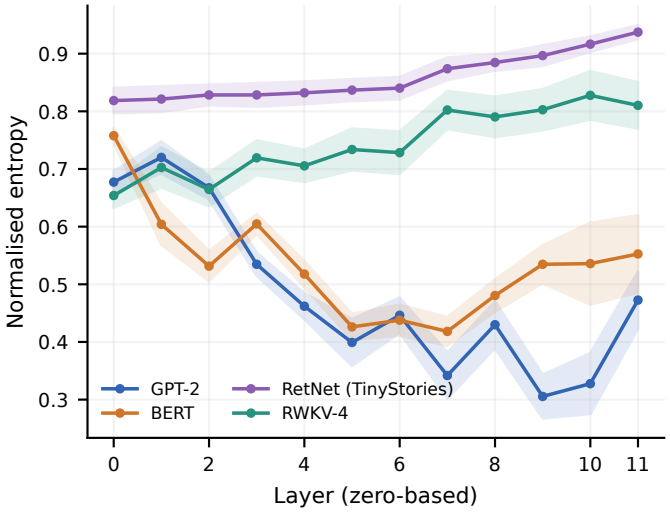



Fig. 4. Normalised entropy by layer: prompt mean with ±1 population SD across 32 prompts, after averaging heads. Causal query zero is excluded. Bands are not confidence intervals; final-layer values are in Table V.

TABLE V
LAYER 11 ONLY: ENTROPY AND FIRST-TOKEN MASS AS MEAN AND POPULATION SD ACROSS PROMPTS. FIRST-TOKEN MASS IS A PERCENTAGE.

| Model | Entropy | First token |
|---|---|---|
| GPT-2 | $0.473 \pm 0.050$ | $67.38 \pm 3.89$ |
| BERT | $0.553 \pm 0.067$ | $3.19 \pm 0.83$ |
| RetNet | $0.937 \pm 0.012$ | $17.79 \pm 3.71$ |
| RWKV-4 | $0.810 \pm 0.040$ | $31.69 \pm 8.35$ |

at layer 0 to 0.840 at layer 6, followed by a larger rise to 0.937 at layer 11. RWKV-4 rises overall from 0.654 to 0.810 with fluctuations. GPT-2 peaks at 0.720 at layer 1, falls to 0.305 at layer 9, then rebounds; BERT falls from 0.758 at layer 0 to 0.418 at layer 7 before recovering to 0.553.

Higher entropy is not evidence of better reasoning, and lower entropy is not evidence of worse reasoning. A head that attends sharply to the first token has low entropy and may be doing nothing useful, and a near-uniform head may be averaging a lot of information.

### D. Head-type distribution

The head-type distributions differ sharply between models. In GPT-2, 106 of 144 heads (73.61%) are first-token heads, and the next largest category is local-window (15 heads, 10.42%). In BERT, 124 heads (86.11%) fall into other, and only 9 are broad. The RetNet checkpoint has only two types: 45 local-window heads (31.25%) and 99 broad heads (68.75%). RWKV-4 is the most mixed, with 30 first-token heads (20.83%), 21 self heads (14.58%), 25 local heads (17.36%), 44 broad heads (30.56%) and 17 in other.

The BERT result needs the most careful reading. A count of 124 in other means those heads did not satisfy any of our rules, which says that the rule set does not describe BERT well. It does not say that those heads do nothing. Bidirectional models spread attention across positions in ways that previous, next, self and first-token tests don't capture, and our classifier has no test for delimiter or [SEP] attention, which earlier work has found to be common in BERT [11]. For RetNet, the absence of first, previous, next and self heads is consistent with the high entropy in Table IV.

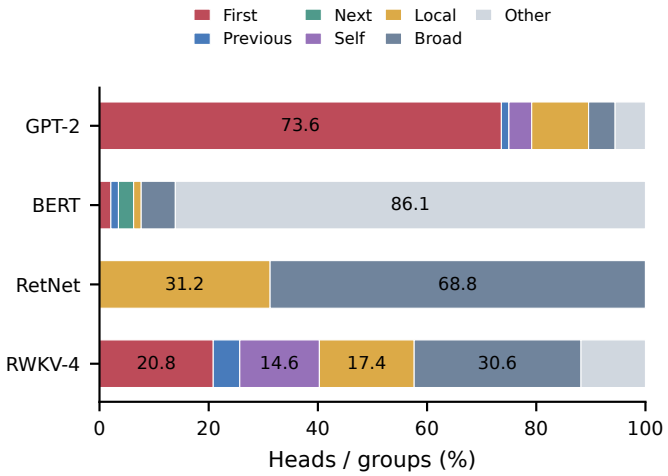



Fig. 5. Head-type percentages under the baseline priority rules. Counts and percentages, including small or zero categories, are in Table VI; RWKV-4 uses pseudo-head groups.

### E. Relative distance by layer and head

Across prompt-averaged cells, GPT-2 ranges from 2.96% (layer 0, head 1) to 48.97% (layer 7, head 2), while BERT ranges from 12.78% to 47.16%. RetNet is much more homogeneous, spanning 20.69–26.56%. RWKV-4 spans 3.07–43.48% and shows a strong group-wise gradient: at layer 11, group 0 has 32.27% distance and group 11 has 3.07%. These are differences between analysis groups, not evidence of native RWKV heads.

A short mean distance means a head keeps its mass near the query token, and a long one means it reaches far. Neither tells us whether a model can handle long-range dependencies. A head with a long mean distance because of a sink, as in GPT-2, is doing something very different from one that spreads mass across the sentence.

### F. First-token mass by layer

At layer 11, first-token mass is 67.38 ± 3.89% for GPT-2, 31.69 ± 8.35% for RWKV-4, 17.79 ± 3.71% for RetNet and 3.19 ± 0.83% for BERT. Compared with the all-layer averages, GPT-2's mass is higher at the end, RWKV's is lower, RetNet's is slightly higher and BERT's is much lower. The BERT value is below the uniform reference of about 4 to 9%, so at the last layer the model attends to [CLS] less than uniform rows would. Since [CLS] is a special token that the model uses in a different way from a text-initial token, the BERT numbers should not be put in the same sentence as GPT-2's as if both described the same phenomenon. RWKV's final-layer standard deviation (8.35 points) is the largest of

TABLE VI
HEAD-TYPE COUNTS AND PERCENTAGES OF 144 HEADS OR GROUPS PER MODEL. EACH ENTRY IS COUNT (PERCENT). ALL CATEGORIES, INCLUDING ZEROS, ARE SHOWN.

| Model | First | Previous | Next | Self | Local | Broad | Other |
|---|---|---|---|---|---|---|---|
| GPT-2 | 106 (73.61) | 2 (1.39) | 0 (0.00) | 6 (4.17) | 15 (10.42) | 7 (4.86) | 8 (5.56) |
| BERT | 3 (2.08) | 2 (1.39) | 4 (2.78) | 0 (0.00) | 2 (1.39) | 9 (6.25) | 124 (86.11) |
| RetNet | 0 (0.00) | 0 (0.00) | 0 (0.00) | 0 (0.00) | 45 (31.25) | 99 (68.75) | 0 (0.00) |
| RWKV-4 | 30 (20.83) | 7 (4.86) | 0 (0.00) | 21 (14.58) | 25 (17.36) | 44 (30.56) | 17 (11.81) |

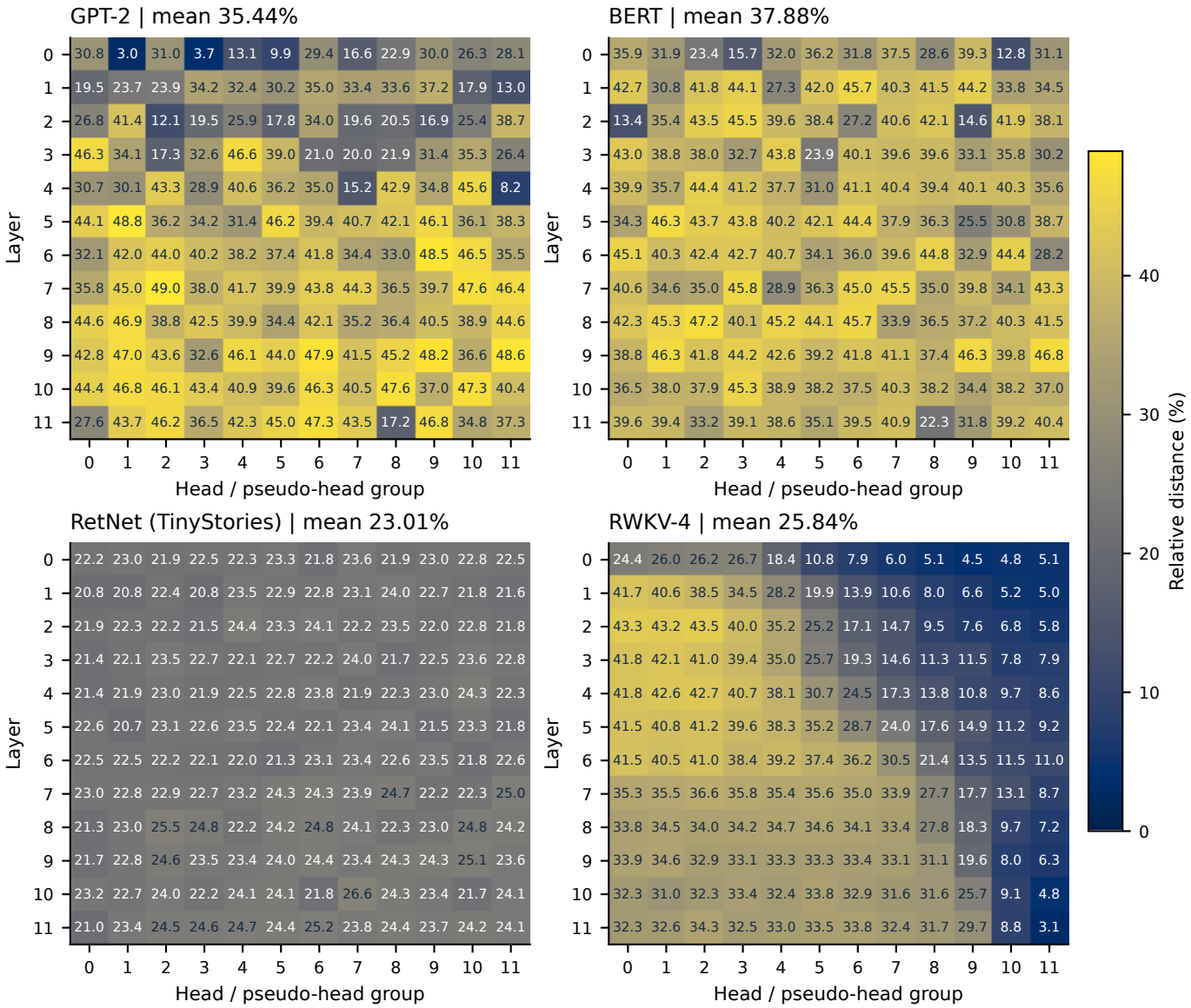



Fig. 6. Prompt-averaged relative distance for every layer and head or group. Cell annotations are percentages; the scale is shared. Distances are tokenizer-relative and do not measure ability on long-range tasks.

the four, so first-token mass in that model varies more from prompt to prompt than in the others.

### G. Lookback offsets

The offset curves show where mass sits relative to the query. Offset zero is self-attention and offset one is the previous token.

For offsets $d = 0, 1, 5$, respectively, the mean masses are GPT-2: 16.82%, 17.80%, 11.90%; BERT: 10.46%, 7.39%, 3.63%; RetNet: 29.26%, 18.41%, 10.90%; RWKV-4: 26.14%, 21.92%, 9.91%. All 32 prompts contribute at these offsets. At $d = 10$, GPT-2, RetNet and RWKV-4 each have 24 eligible prompts, while BERT has 32. The complete values and eligible counts for $d = 0, \ldots, 10$ are in Appendix F.

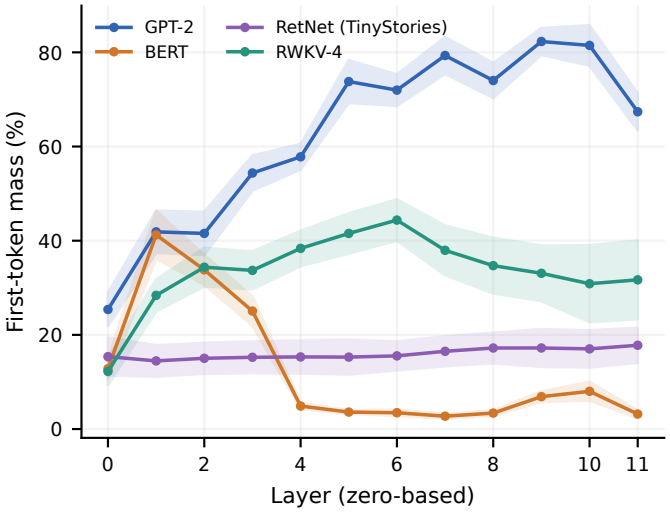



Fig. 7. First-token mass by layer: mean and ±1 population SD over prompts. Query zero is omitted. BERT position zero is [CLS], while causal models use the first text token. Final-layer values are in Table V.

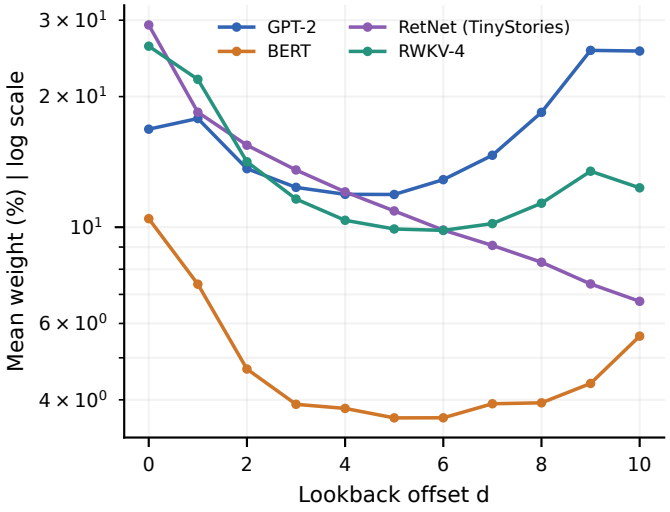



Fig. 8. Lookback weight on a logarithmic scale. Each prompt contributes its mean over eligible queries, heads and layers, then eligible prompts are weighted equally. Only past/current offsets are shown, including for BERT. Appendix F gives all values and eligible counts.

The curves do not fall steadily with offset. GPT-2 and RWKV both rise again at larger offsets on this prompt set. In GPT-2 this is what we would expect from the first-token sink, because for a query at position i the first token sits at offset i, so sink mass shows up at larger offsets as i grows. Larger offsets also have fewer eligible prompts, since a prompt needs at least d + 1 tokens, so their estimates are noisier. We therefore avoid reading specific bumps in the tail as properties of the models.

### H. Supplementary views

The PCA head atlas, per-model layer snapshots, entropy heatmap, head-type map, offset profile, distance heatmap, sink-by-layer view and entropy bands are included in Appendix C.

TABLE VII
SELECTED SENSITIVITY COUNTS. APPENDIX B GIVES ALL SEVEN TYPES FOR WINDOWS 1, 3 AND 5.

| Model | Delta | Type | Count |
|---|---|---|---|
| GPT-2 | −0.1 | First | 114 |
| GPT-2 | +0.1 | First / Other | 89 / 38 |
| BERT | −0.1 | Broad | 26 |
| BERT | +0.1 | Other | 136 |
| RetNet | −0.1 | Local | 144 |
| RetNet | +0.1 | Broad / Other | 29 / 115 |
| RWKV-4 | −0.1 | First / Other | 57 / 0 |
| RWKV-4 | +0.1 | First / Other | 25 / 58 |

The atlas is separate from the explorer head-comparison grid; see Fig. 9 in the appendix.

### I. Threshold sensitivity

The classifier in Section IV-G uses hand-chosen thresholds, so we tested how much the head-type counts depend on them. We shifted all thresholds together by -0.1, 0 and +0.1, which includes the broad-entropy threshold, and kept the local window at ±3. This is not a one-threshold-at-a-time experiment. We also ran windows of 1 and 5, and all results are in outputs/metrics/sensitivity/results.json.

The counts move a lot. For GPT-2 a shift of -0.1 raises first-token heads from 106 to 114, and a shift of +0.1 lowers them to 89 and pushes 38 heads into other (up from 8). BERT shows the same kind of movement. At -0.1, 26 heads become broad and at +0.1 the other class grows to 136. RetNet is the most sensitive of the four. At -0.1, all 144 heads are labelled local-window. At baseline, 45 are local and 99 are broad. At +0.1, 29 are broad and 115 are other. This pattern is consistent with a model whose heads sit near the boundaries of our rules and not near a clear cluster. RWKV is also sensitive: first-token heads fall from 57 to 30 to 25 across the three settings, and other grows from 0 to 17 to 58.

Two conclusions survive the sweep. GPT-2 has by far the largest number of first-token heads in all three settings, and BERT has by far the largest number of other heads. The finer statements, for example that RetNet has 99 broad heads, depend on the threshold and should be quoted only with it.

## VII. DISCUSSION

### A. What the comparison shows

The clearest difference is the first-token sink. GPT-2 puts most of its attention mass on position zero, in a way that is visible in the head-type counts, in the all-layer and final-layer sink numbers, and in the offset curves. RWKV-4 has a smaller version of the same behaviour. The RetNet checkpoint has sink mass close to what uniform rows would produce, and BERT, at the final layer, has less mass on [CLS] than uniform rows would. A description that fits all four is that autoregressive models trained to convergence on large corpora tend to develop a sink, and that our short-trained RetNet has not. The data in this study are compatible with that description but cannot establish it. We have one checkpoint per model, and RetNet differs from the others in many ways besides training length.

### B. Entropy and distance together

Taken together, entropy and distance give a more useful picture than either alone. GPT-2 combines low entropy with a relative distance well above the causal uniform value, which is the signature of a sink and not of long-range routing. RetNet combines high entropy with a distance slightly below the uniform value. This is what we would expect if most heads spread mass broadly with a mild preference for nearby tokens, as a decay term would encourage. The explanation is a hypothesis. Retention includes a fixed exponential decay by design [7], so some recency preference is built in, and a very short training run may have left the learned query-key part close to its initialisation. We did not test either idea. A natural test would be to train the same RetNet for longer and track how the entropy and sink metrics change.

### C. Why BERT looks different

BERT differs from the other three in more than one way: it is bidirectional, it uses [CLS] and [SEP], and it was trained with masked language modelling. Each of these changes what our metrics mean. Relative distance has a different uniform reference, first-token mass refers to a special token, and offsets exclude forward-looking weights. The large other class reflects the limits of our rules more than anything about BERT's heads. Prior work on BERT found that many heads concentrate on [SEP], [CLS] or punctuation [11], and our classifier cannot name these patterns. We would expect an extended rule set with delimiter tests to reduce the other count, and we flag this as future work.

### D. RWKV pseudo-heads

RWKV-4's profile is the most mixed, and the explanation may be partly methodological. Averaging 64 channels into one pseudo-head blends channels that have different decay rates. Channels with a fast decay concentrate on recent tokens, and channels with a slow decay spread out. An average over a group can therefore look broad even if each member is sharp, or can look local if fast channels dominate. A different grouping, for example by decay rate, could give different head-type counts. The reconstruction also leaves out token shift and the receptance gate, so these pseudo-heads are a partial picture of what RWKV mixes.

### E. What is a finding and what is a hypothesis

We list the status of the main statements to keep the two separate.

Findings, for these checkpoints and prompts: the metric values in Tables IV–VI; GPT-2's large first-token mass; BERT's low final-layer [CLS] mass; RetNet's high entropy and short relative distance; the sensitivity of head-type counts to thresholds.

Hypotheses, not tested here: that the RetNet profile reflects its short training; that the sink reflects a training dynamic shared by autoregressive models; that the BERT other class would shrink under a richer rule set; that grouping by decay rate would change the RWKV profile.

### F. Using the toolkit

A researcher who wants to try an idea like these hypotheses can load saved records, change the thresholds or window, and regenerate every figure with the scripts. The interface is meant for the earlier step of looking at individual prompts, where unusual rows show up before they are quantified. Several of the patterns we describe (the sink, the near-uniform RetNet rows) are visible in the heatmap without computing anything.

## VIII. Limitations

1) Small and narrow prompt set. We used 32 short, hand-written English prompts. There is no long-context evaluation, no other language and no domain-specific text. The results do not establish general behaviour.
2) Confounded models. The four checkpoints differ in training corpus, objective, tokenisation, size and maturity. GPT-2 and BERT are fully trained models, RWKV-4 was trained on the Pile, and our RetNet has an author-reported 1,000-step TinyStories run, with conflicting 100-step metadata and loss records (Section V-B). Every difference between models in this paper could come from any of these factors, and we cannot separate them. The study does not isolate architectural effects.
3) Early RetNet checkpoint. The RetNet in this study is not a matched pretrained comparison. Its metrics should be read as describing an early-training state. The supplied notebook identifies a Tesla T4, seed 42 and optimiser/schedule code, but its saved output and metadata disagree with the author-confirmed run length and validation-batch count. Exact historical library versions and a matching checkpoint execution record are missing.
4) Different semantics. Softmax weights are probabilities that mix value vectors. Retention scores are signed and our use of absolute values discards cancellation. RWKV weights come from a partial reconstruction. Treating these as the same object is a convention that makes comparison possible and does not mean they have the same meaning.
5) Arbitrary RWKV grouping. Twelve groups of 64 contiguous channels is our choice. A different grouping would give different head-level results.
6) Special tokens. BERT's first position is [CLS], and [SEP] carries mass in ways we do not track separately, so first-token comparisons with GPT-2 and the others are not like for like.
7) Rollout. The rollout used here is a descriptive proxy. In the current implementation the residual weight for RetNet depends on input scale (Section IV-D).
8) Statistics. The standard-deviation bands show prompt-to-prompt variation. They are not confidence intervals. We did not run significance tests and did not perform any intervention, so nothing in this paper bears on whether any head is causally important.
9) Threshold sensitivity. Head labels depend on hand-chosen thresholds, and Section VI-I shows that counts change substantially when they move. The other class

reflects limited rule coverage and not absence of meaningful behaviour. The classifier does not detect induction, syntactic or delimiter heads.

10) Float16 storage. Tensors are stored in half precision. We have not measured the effect of this on the metrics, and expect it to be small but have not shown it.

## IX. Conclusion

We built AttentionLens to make attention and token-mixing patterns from GPT-2, BERT, RetNet and RWKV-4 comparable under a single tensor format, one set of metrics and one documented procedure for each choice that could change the numbers. On 32 prompts and 576 head instances, the toolkit produced six figures and five tables that can be regenerated from saved records.

The observations we are most confident about are narrow. GPT-2 places a majority of its attention on the first token and most of its heads are first-token heads. BERT puts little mass on [CLS] at the final layer, and most of its heads do not match any of our positional rules. The RetNet checkpoint has the highest entropy and the shortest relative distance of the four, with sink mass close to what uniform rows would give. RWKV-4 sits in between, with a mixed head-type profile. Head-type counts shift by tens of heads when thresholds change by 0.1, so category counts should always be quoted together with the thresholds that produced them.

Several directions would make the comparison stronger. Using a fully trained RetNet, or training the same model for longer and tracking its metrics, would show how much of the RetNet profile comes from maturity. A larger and more varied prompt set, including long contexts, would test whether the patterns hold outside short sentences. Extending the classifier with delimiter and induction tests, and replacing the contiguous RWKV grouping with a grouping by decay rate, would reduce two of the largest sources of ambiguity in Section VIII. Interventions such as head ablation and patching would be needed before any head could be called important for a prediction. We also plan to add more architectures, including state space models, through the same extractor registry.

## Code and Data Availability

Repository:
https://github.com/mitravarun123/Attention-Visualisation-

Licence:

No project-level licence file was found in the supplied folder. Public hosting and release status remain to be confirmed; this paper does not assert an open-source release.

Configurations and prompts: `configs/prompts.yaml`; sensitivity results: `outputs/metrics/sensitivity/results.json`. Figures: PDF, 400-dpi PNG and CSV for each of the six main figures in `outputs/paper_figures`; supplementary figures in `outputs/figures`. Model identifiers: gpt2, bert-base-uncased, `RWKV/rwkv-4-169m-pile`, and the RetNet checkpoint at `checkpoints/retnet_128.pt` with its `training_metadata.json`.

The checkpoint and metadata are present in the supplied local project. Their public release location, revision identifiers and correspondence to the author-confirmed training run remain to be established.

Reproduction commands:

```
source .venv/bin/activate
export HF_HOME="$PWD/.cache/huggingface"
export MPLCONFIGDIR="$PWD/.cache/matplotlib"
python scripts/extract_all.py gpt2 bert retnet rwkv
python scripts/compute_metrics.py
python scripts/make_figures.py
python scripts/make_analysis_figures.py
python scripts/sensitivity.py
python scripts/make_paper_figures.py
python scripts/run_web.py
```

Current inspected versions are recorded in `data/environment-inspected.json` in the paper source package; the historical extraction/training environment remains unrecorded.

.

## Appendix A
## Full Prompts

The following strings are reproduced exactly from `configs/prompts.yaml`. Token counts, measured from the saved tensors, are listed in GPT-2 / BERT / RetNet / RWKV-4 order. The strings are not balanced experimental categories.

1) The quick brown fox jumps over the lazy dog. *Tokens: 10 / 12 / 10 / 10.*
2) Attention is all you need for sequence modelling. *Tokens: 10 / 11 / 10 / 10.*
3) The capital of France is Paris, and the capital of Germany is Berlin. *Tokens: 15 / 17 / 15 / 15.*
4) She opened the old book and began to read the first chapter aloud. *Tokens: 14 / 16 / 14 / 14.*
5) In 2017, researchers introduced the transformer architecture. *Tokens: 9 / 12 / 9 / 9.*
6) The model predicts the next token given all previous tokens. *Tokens: 11 / 15 / 11 / 11.*
7) After the meeting, he walked home because the weather was nice. *Tokens: 13 / 15 / 13 / 13.*
8) Neural networks learn representations from large amounts of data. *Tokens: 11 / 12 / 11 / 11.*
9) My name is Mitra Varun I am a very good boy, I used to work on Artificial Intelligence projects. *Tokens: 23 / 25 / 23 / 24.*
10) Artificial Intelligence is very powerful technology, AI changes many other fields. *Tokens: 14 / 15 / 14 / 14.*
11) The cat that the dog chased climbed the tall tree. *Tokens: 11 / 13 / 11 / 11.*
12) Alice gave Bob a book because he wanted to learn astronomy. *Tokens: 12 / 14 / 12 / 13.*
13) Although it was raining, the children continued playing outside. *Tokens: 11 / 13 / 11 / 12.*
14) If the temperature falls below zero, the water will freeze. *Tokens: 12 / 14 / 12 / 12.*
15) First wash the vegetables, then chop them, and finally cook them. *Tokens: 14 / 16 / 14 / 14.*
16) The red box is beside the blue box, which is under the table. *Tokens: 15 / 17 / 15 / 15.*
17) One, two, three, four, five, six, seven, eight, nine, ten. *Tokens: 20 / 22 / 20 / 20.*
18) The word echo appears again: echo, echo, echo. *Tokens: 12 / 14 / 12 / 12.*
19) A small bird sang while a large bird watched from the roof. *Tokens: 13 / 15 / 13 / 13.*
20) Neither the teacher nor the students knew the answer. *Tokens: 10 / 12 / 10 / 10.*
21) What did the scientist discover after repeating the experiment? *Tokens: 10 / 12 / 10 / 10.*
22) Please close the window before you leave the room. *Tokens: 10 / 12 / 10 / 10.*
23) I thought the train would arrive early, but it was delayed. *Tokens: 13 / 15 / 13 / 13.*
24) The painter mixed blue and yellow to make green. *Tokens: 10 / 12 / 10 / 10.*
25) Yesterday we visited the museum; tomorrow we will visit the library. *Tokens: 13 / 15 / 13 / 13.*
26) The password contains letters, numbers, and punctuation marks. *Tokens: 12 / 15 / 12 / 12.*
27) Python functions can accept arguments and return computed values. *Tokens: 10 / 12 / 10 / 10.*
28) For every action, there is an equal and opposite reaction. *Tokens: 12 / 14 / 12 / 12.*
29) The river flows through the valley and reaches the sea. *Tokens: 11 / 13 / 11 / 11.*
30) He said, 'Bring the notebook,' and she replied, 'I already have it.' *Tokens: 18 / 22 / 18 / 18.*
31) The first key opens the door; the second key opens the cabinet. *Tokens: 14 / 16 / 14 / 14.*
32) At the end of the long journey, the travellers returned home. *Tokens: 13 / 15 / 13 / 13.*

## Appendix B
## Complete Sensitivity Tables

Each row reclassifies 144 heads or groups. All thresholds, including broad entropy, are shifted together by $\delta$ and clamped to $[0, 1]$; the displayed window radius is fixed within each table. All 36 stored runs were independently recomputed from the saved tensors with the supplied classifier and matched exactly.

TABLE VIII
COMPLETE SENSITIVITY COUNTS WITH WINDOW RADIUS 1.

| Model | $\delta$ | First | Previous | Next | Self | Local | Broad | Other |
|---|---|---|---|---|---|---|---|---|
| GPT-2 | $-0.1$ | 114 | 5 | 0 | 6 | 0 | 16 | 3 |
| GPT-2 | $+0.0$ | 106 | 2 | 0 | 6 | 0 | 8 | 22 |
| GPT-2 | $+0.1$ | 89 | 1 | 0 | 4 | 0 | 3 | 47 |
| BERT | $-0.1$ | 11 | 5 | 5 | 2 | 0 | 26 | 95 |
| BERT | $+0.0$ | 3 | 2 | 4 | 0 | 0 | 9 | 126 |
| BERT | $+0.1$ | 2 | 1 | 2 | 0 | 0 | 1 | 138 |
| RetNet | $-0.1$ | 0 | 0 | 0 | 0 | 0 | 144 | 0 |
| RetNet | $+0.0$ | 0 | 0 | 0 | 0 | 0 | 144 | 0 |
| RetNet | $+0.1$ | 0 | 0 | 0 | 0 | 0 | 29 | 115 |
| RWKV-4 | $-0.1$ | 57 | 10 | 0 | 32 | 2 | 43 | 0 |
| RWKV-4 | $+0.0$ | 30 | 7 | 0 | 21 | 6 | 60 | 20 |
| RWKV-4 | $+0.1$ | 25 | 3 | 0 | 4 | 10 | 16 | 86 |

TABLE IX
COMPLETE SENSITIVITY COUNTS WITH WINDOW RADIUS 3.

| Model | $\delta$ | First | Previous | Next | Self | Local | Broad | Other |
|---|---|---|---|---|---|---|---|---|
| GPT-2 | $-0.1$ | 114 | 5 | 0 | 6 | 18 | 1 | 0 |
| GPT-2 | $+0.0$ | 106 | 2 | 0 | 6 | 15 | 7 | 8 |
| GPT-2 | $+0.1$ | 89 | 1 | 0 | 4 | 9 | 3 | 38 |
| BERT | $-0.1$ | 11 | 5 | 5 | 2 | 4 | 26 | 91 |
| BERT | $+0.0$ | 3 | 2 | 4 | 0 | 2 | 9 | 124 |
| BERT | $+0.1$ | 2 | 1 | 2 | 0 | 2 | 1 | 136 |
| RetNet | $-0.1$ | 0 | 0 | 0 | 0 | 144 | 0 | 0 |
| RetNet | $+0.0$ | 0 | 0 | 0 | 0 | 45 | 99 | 0 |
| RetNet | $+0.1$ | 0 | 0 | 0 | 0 | 0 | 29 | 115 |
| RWKV-4 | $-0.1$ | 57 | 10 | 0 | 32 | 25 | 20 | 0 |
| RWKV-4 | $+0.0$ | 30 | 7 | 0 | 21 | 25 | 44 | 17 |
| RWKV-4 | $+0.1$ | 25 | 3 | 0 | 4 | 38 | 16 | 58 |

TABLE X
COMPLETE SENSITIVITY COUNTS WITH WINDOW RADIUS 5.

| Model | $\delta$ | First | Previous | Next | Self | Local | Broad | Other |
|---|---|---|---|---|---|---|---|---|
| GPT-2 | $-0.1$ | 114 | 5 | 0 | 6 | 19 | 0 | 0 |
| GPT-2 | $+0.0$ | 106 | 2 | 0 | 6 | 30 | 0 | 0 |
| GPT-2 | $+0.1$ | 89 | 1 | 0 | 4 | 23 | 3 | 24 |
| BERT | $-0.1$ | 11 | 5 | 5 | 2 | 37 | 14 | 70 |
| BERT | $+0.0$ | 3 | 2 | 4 | 0 | 12 | 8 | 115 |
| BERT | $+0.1$ | 2 | 1 | 2 | 0 | 2 | 1 | 136 |
| RetNet | $-0.1$ | 0 | 0 | 0 | 0 | 144 | 0 | 0 |
| RetNet | $+0.0$ | 0 | 0 | 0 | 0 | 144 | 0 | 0 |
| RetNet | $+0.1$ | 0 | 0 | 0 | 0 | 143 | 1 | 0 |
| RWKV-4 | $-0.1$ | 57 | 10 | 0 | 32 | 45 | 0 | 0 |
| RWKV-4 | $+0.0$ | 30 | 7 | 0 | 21 | 78 | 1 | 7 |
| RWKV-4 | $+0.1$ | 25 | 3 | 0 | 4 | 54 | 3 | 55 |

## APPENDIX C
## ADDITIONAL FIGURES

These complementary views use the same saved records. This visual appendix uses full-width plates for legibility. The PCA atlas projects eleven standardised prompt-averaged features and is distinct from the explorer head-comparison grid. Layer snapshots illustrate the first prompt rather than dataset means.

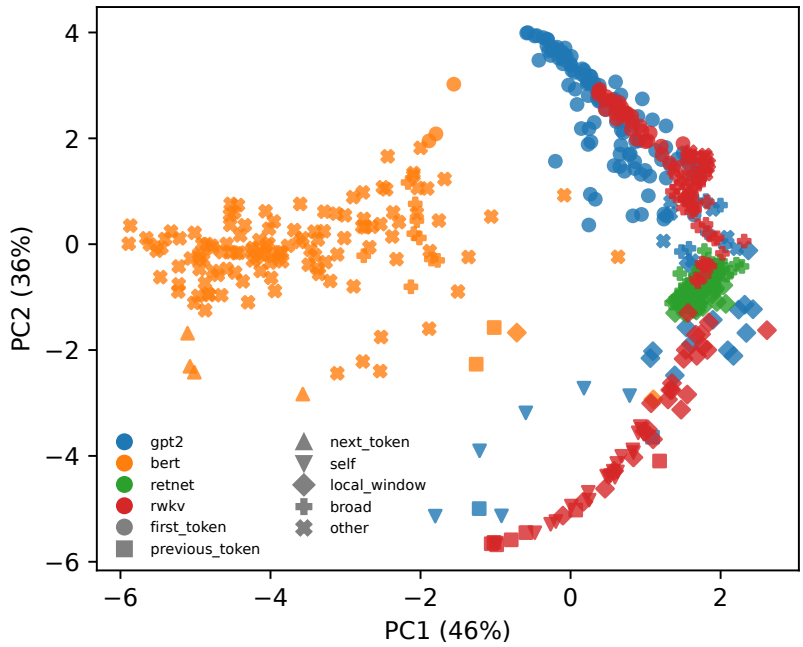



Fig. 9. PCA atlas of 576 heads or groups. The first two components explain about 46% and 36% of feature variance. Colours encode models and marker shapes encode positional labels. PCA geometry is descriptive and does not establish head function.

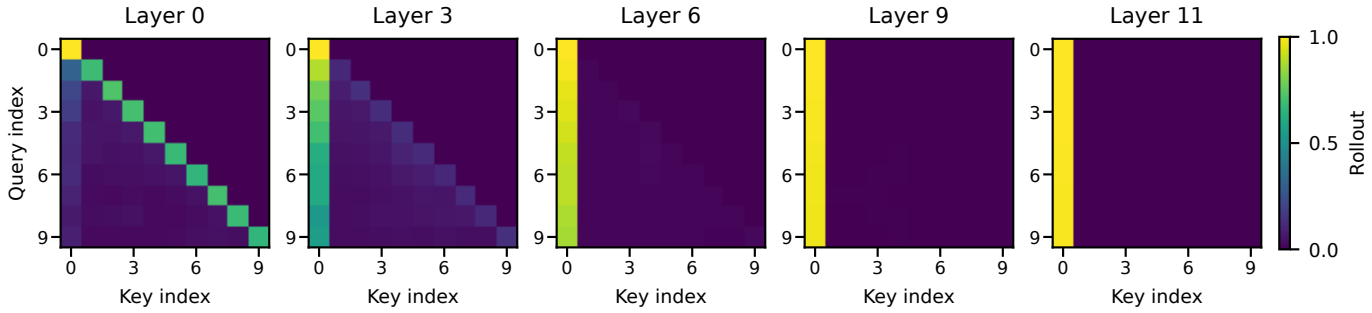



Fig. 10. GPT-2 cumulative rollout for the first shared prompt at layers 0, 3, 6, 9 and 11, with one linear $[0, 1]$ colour scale. Axes are token indices using the model-specific sequence in Fig. 3; these are illustrative input-specific views.

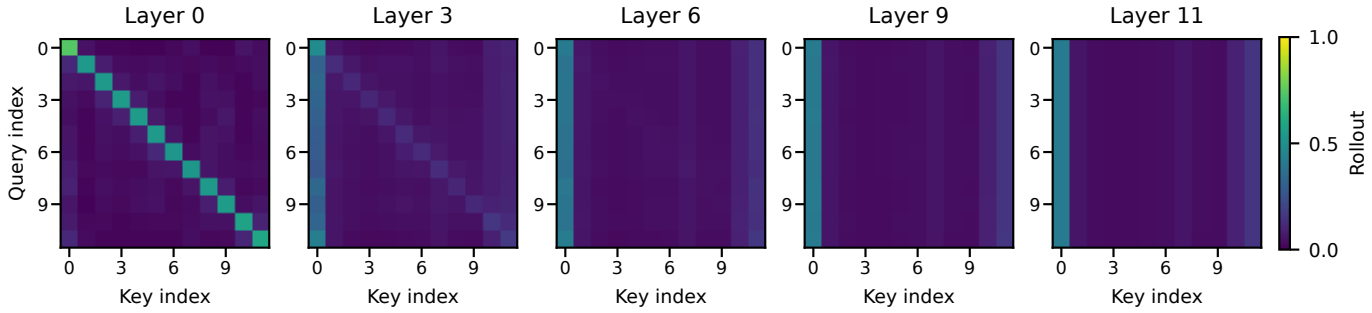



Fig. 11. BERT cumulative rollout for the first shared prompt at layers 0, 3, 6, 9 and 11, with one linear $[0, 1]$ colour scale. Axes are token indices using the model-specific sequence in Fig. 3; these are illustrative input-specific views.

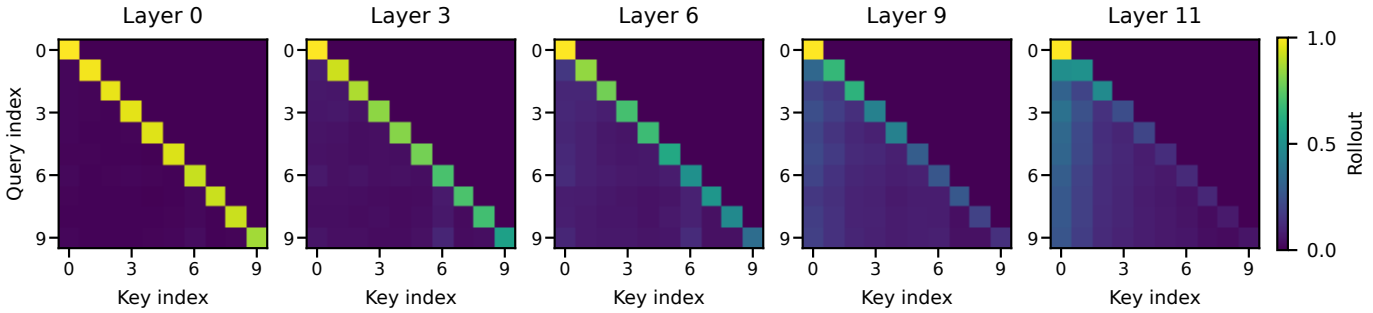



Fig. 12. RetNet cumulative rollout for the first shared prompt at layers 0, 3, 6, 9 and 11, with one linear $[0, 1]$ colour scale. Axes are token indices using the model-specific sequence in Fig. 3; these are illustrative input-specific views.

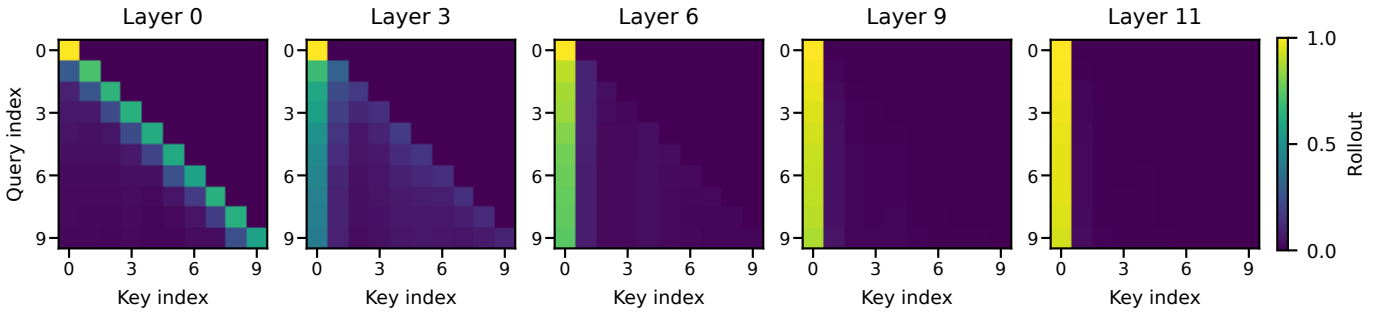



Fig. 13. RWKV-4 cumulative rollout for the first shared prompt at layers 0, 3, 6, 9 and 11, with one linear $[0, 1]$ colour scale. Axes are token indices using the model-specific sequence in Fig. 3; these are illustrative input-specific views.

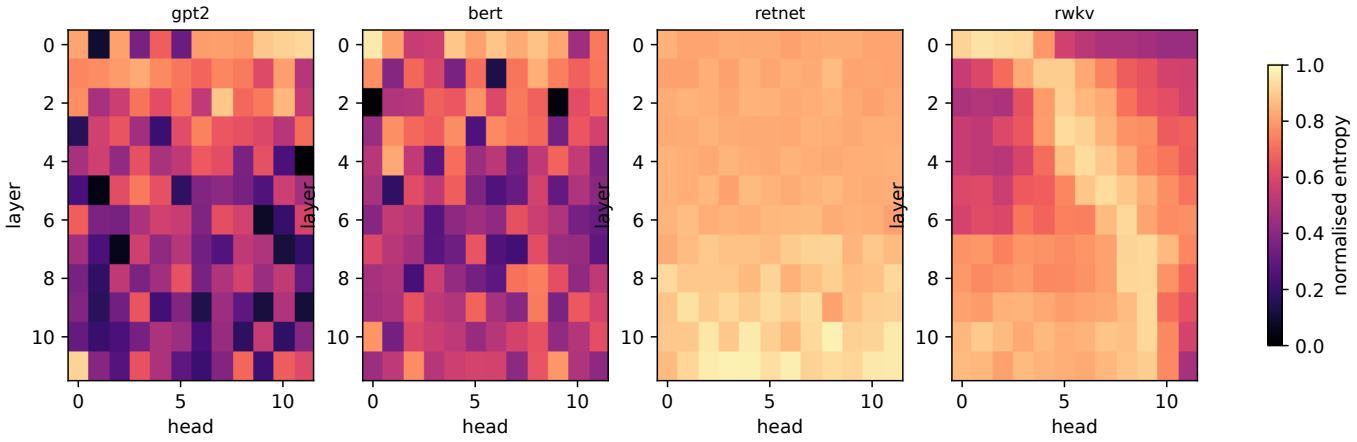



Fig. 14. Prompt-averaged normalised entropy by layer and head or group, using a shared colour scale.

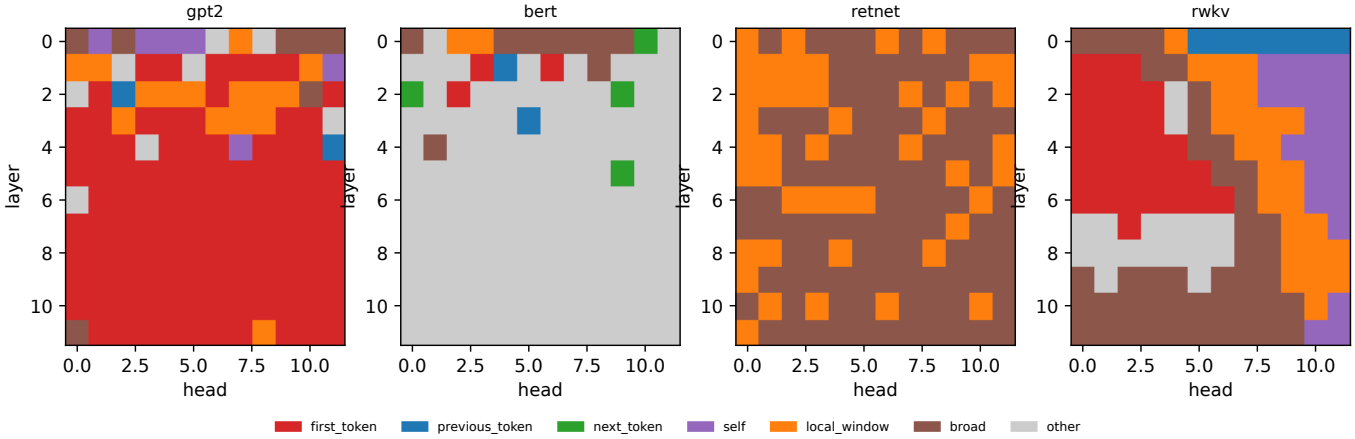



Fig. 15. Head-type map under the baseline priority rules. Colour categories are positional descriptions rather than demonstrated causal functions.

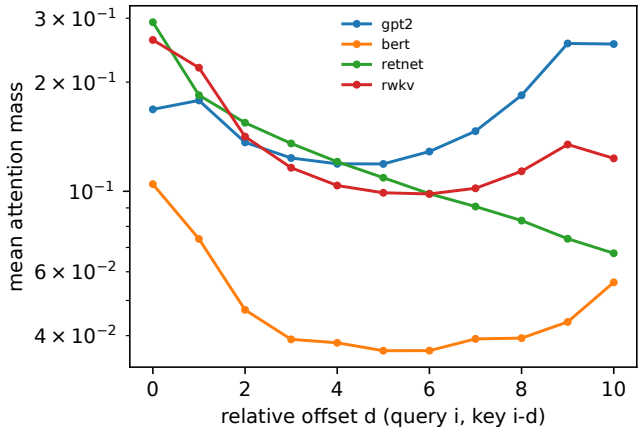



Fig. 16. Alternate lookback-offset view of the same means as Fig. 8; only past/current offsets are included.

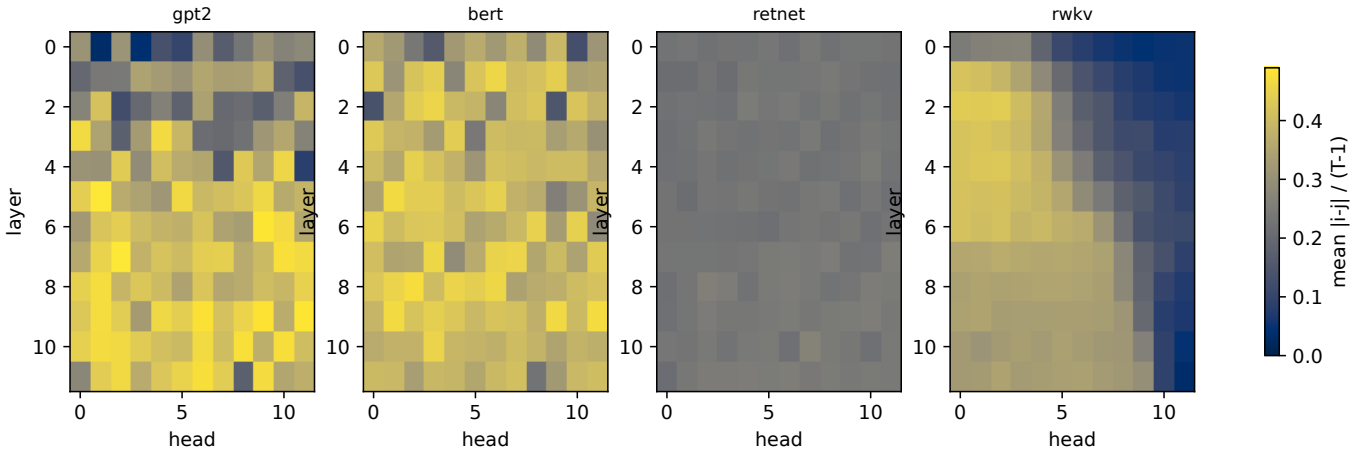



Fig. 17. Alternate unannotated relative-distance heatmaps of the values in Fig. 6, with a shared colour scale.

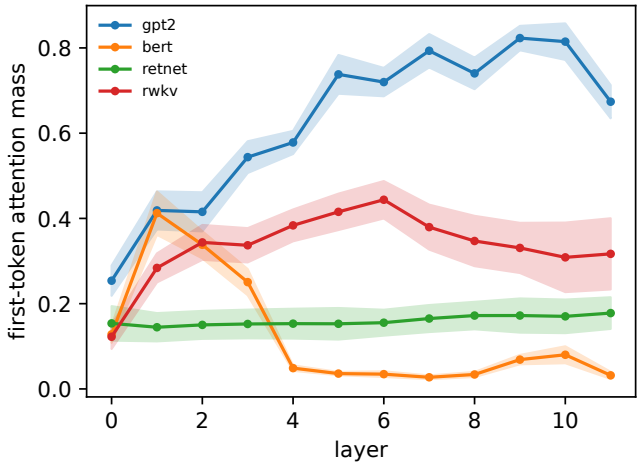



Fig. 18. First-token mass with population SD bands, an alternate view of Fig. 7; BERT token zero is [CLS].

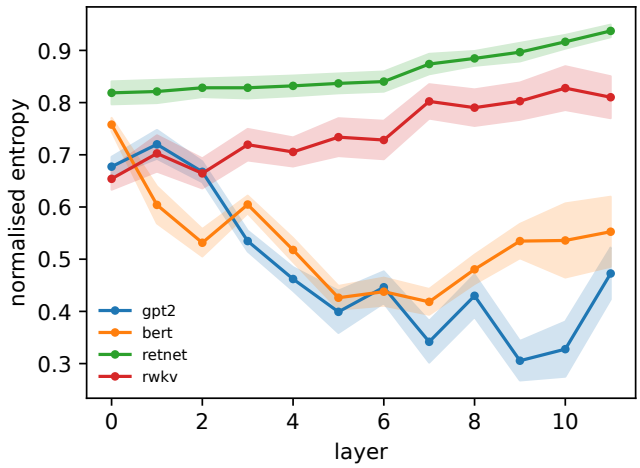



Fig. 19. Normalised entropy with population SD bands, an alternate view of Fig. 4; bands describe prompt variation.

## APPENDIX D
## TRAINING AND ENVIRONMENT DETAILS

The author-confirmed settings are 1,000 steps and 100 validation batches. The supplied notebook code uses AdamW, peak learning rate $3 \times 10^{-4}$, weight decay 0.1, betas $(0.9, 0.95)$, seed 42, batch size 1, accumulation 8, sequence length 128, dropout 0.1 and gradient clipping at 1.0. It uses float16 autocast and `GradScaler` on a Tesla T4. The written 1,000-step schedule has 100 linear-warmup steps and cosine decay thereafter to a tenth of the peak rate. This describes the supplied code and the author's corrected configuration, not a newly executed training run.

The notebook streams TinyStories, concatenates GPT-2 token IDs with an EOS marker per story, and forms non-overlapping chunks of 129 tokens to create 128 input–target pairs. Validation code takes the initial batches of the validation stream. With 100 full batches of size 1 and 128 targets per batch, the confirmed configuration would evaluate 12,800 target positions. The archived validation output instead reports 10 batches, nominally 1,280 targets. A corrected execution record is needed to associate the reported validation loss with either sample size.

The exported metadata records 100 optimiser steps, final training loss 4.515007555484772 and validation loss 4.434524583816528. Saved output prints a 100-step learning-rate schedule and the same rounded losses, despite the current configuration cell specifying 1,000 steps. The notebook's source and output therefore cannot be treated as a single internally consistent execution trace. The paper uses the author's corrected training description and retains this discrepancy explicitly. Checkpoint maturity remains a confound, and the recorded losses should not be relabelled as verified 1,000-step losses.

The inspected local environment versions are listed in Section V-D and `data/environment-inspected.json`. They do not establish historical Colab package versions or original extraction hardware. No project-level licence file or public-release manifest was found in the supplied directory.

## APPENDIX E
## EXTENDED EQUATIONS

For one RWKV-4 channel, let $\lambda_c = \exp(\theta_c) > 0$, where $\theta_c$ is the checkpoint's `time_decay`; the implementation uses $-\lambda_c$ in the logits. To avoid confusing the accumulator with the explicit token coefficient, denote numerator and denominator states by $\alpha_{t,c}$ and $\beta_{t,c}$ (often written $a_t, b_t$). With zero initial states,

$$\alpha_{t,c} = e^{-\lambda_c}\alpha_{t-1,c} + e^{k_{t,c}} v_{t,c}, \tag{13}$$

$$\beta_{t,c} = e^{-\lambda_c}\beta_{t-1,c} + e^{k_{t,c}}. \tag{14}$$

Before updating at position $t$, the output is

$$\mathrm{wkv}_{t,c} = \frac{\alpha_{t-1,c} + e^{u_c + k_{t,c}} v_{t,c}}{\beta_{t-1,c} + e^{u_c + k_{t,c}}}. \tag{15}$$

Expanding the previous state yields

$$\alpha_{t-1,c} = \sum_{i=0}^{t-1} e^{-(t-1-i)\lambda_c + k_{i,c}} v_{i,c}, \tag{16}$$

with an analogous expression for $\beta$ without $v$. Thus the explicit coefficients are $a_{t,i,c} = z_{t,i,c} / \sum_{j=0}^{t} z_{t,j,c}$, where $z$ is the piecewise expression in Section IV-B with $w_c = \lambda_c$. The extractor computes a softmax of these logits, then averages each consecutive block of 64 channels. This expansion is an algebraic description of the WKV weights; it excludes the surrounding token shift, receptance and output projection.

For uniform causal rows, $p_{ij} = 1/(i+1)$ for $j \le i$. The mean distance in row $i$ is

$$\sum_{j=0}^{i} \frac{i-j}{i+1} = \frac{i}{2}. \tag{17}$$

Summing rows and dividing by $T(T-1)$ gives $D = 1/4$ for $T > 1$. With bidirectional uniform rows, the identity

$$\sum_{i,j=0}^{T-1} |i-j| = 2\sum_{d=1}^{T-1} d(T-d) = \frac{T(T^2-1)}{3} \tag{18}$$

gives $D = (T+1)/(3T)$ after accounting for $p_{ij} = 1/T$ and the metric denominator. Uniform causal first-token mass is

$$S = \frac{1}{T-1}\sum_{i=1}^{T-1} \frac{1}{i+1} = \frac{H_T - 1}{T-1}, \tag{19}$$

whereas bidirectional uniform mass is $1/T$. Uniform row entropy is $\log n_i$, so normalised entropy is one when $n_i > 1$. These formulas match direct evaluations of the supplied metric functions for $T = 2, \ldots, 25$; their one-token convention is zero.

## APPENDIX F
## NUMERICAL LOOKBACK PROFILES

The table gives mean percentage weight and eligible prompt count for every plotted offset. Each prompt is averaged over its own valid queries before averaging prompts, so these are not pooled-query estimates.

TABLE XI
LOOKBACK MEAN WEIGHT IN PERCENT (ELIGIBLE PROMPTS), FOR ALL PLOTTED OFFSETS.

| Offset | GPT-2 | BERT | RetNet | RWKV-4 |
|---|---|---|---|---|
| 0 | 16.82 (32) | 10.46 (32) | 29.26 (32) | 26.14 (32) |
| 1 | 17.80 (32) | 7.39 (32) | 18.41 (32) | 21.92 (32) |
| 2 | 13.64 (32) | 4.71 (32) | 15.45 (32) | 14.15 (32) |
| 3 | 12.35 (32) | 3.91 (32) | 13.55 (32) | 11.61 (32) |
| 4 | 11.91 (32) | 3.82 (32) | 12.06 (32) | 10.37 (32) |
| 5 | 11.90 (32) | 3.63 (32) | 10.90 (32) | 9.91 (32) |
| 6 | 12.87 (32) | 3.64 (32) | 9.85 (32) | 9.83 (32) |
| 7 | 14.65 (32) | 3.92 (32) | 9.08 (32) | 10.19 (32) |
| 8 | 18.40 (32) | 3.94 (32) | 8.30 (32) | 11.36 (32) |
| 9 | 25.56 (31) | 4.36 (32) | 7.40 (31) | 13.46 (31) |
| 10 | 25.47 (24) | 5.61 (32) | 6.75 (24) | 12.32 (24) |

## REFERENCES


[1] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," International Conference on Learning Representations, 2015, 2015. [Online]. Available: https://arxiv.org/abs/1409.0473

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423/

[4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI, Tech. Rep., 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[5] J. Vig, "A multiscale visualization of attention in the transformer model," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, M. R. Costa-jussà and E. Alfonseca, Eds. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 37–42. [Online]. Available: https://aclanthology.org/P19-3007/

[6] B. Hoover, H. Strobelt, and S. Gehrmann, "exBERT: A Visual Analysis Tool to Explore Learned Representations in Transformer Models," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, A. Celikyilmaz and T.-H. Wen, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 187–196. [Online]. Available: https://aclanthology.org/2020.acl-demos.22/

[7] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei, "Retentive Network: A Successor to Transformer for Large Language Models," arXiv:2307.08621, 2023. [Online]. Available: https://arxiv.org/abs/2307.08621

[8] B. Peng, E. Alcaide, Q. Anthony, A. Albalak, S. Arcadinho, S. Biderman, H. Cao, X. Cheng, M. Chung, L. Derczynski, X. Du, M. Grella, K. Gv, X. He, H. Hou, P. Kazienko, J. Kocon, J. Kong, B. Koptyra, H. Lau, J. Lin, K. S. I. Mantri, F. Mom, A. Saito, G. Song, X. Tang, J. Wind, S. Woźniak, Z. Zhang, Q. Zhou, J. Zhu, and R.-J. Zhu, "RWKV: Reinventing RNNs for the transformer era," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 14 048–14 077. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.936/

[9] A. Ali, I. Zimerman, and L. Wolf, "The Hidden Attention of Mamba Models," arXiv:2403.01590, 2024. [Online]. Available: https://arxiv.org/abs/2403.01590

[10] I. Zimerman, A. Ali, and L. Wolf, "Explaining Modern Gated-Linear RNNs via a Unified Implicit Attention Formulation," arXiv:2405.16504, 2024. [Online]. Available: https://arxiv.org/abs/2405.16504

[11] K. Clark, U. Khandelwal, O. Levy, and C. D. Manning, "What does BERT look at? an analysis of BERT's attention," in *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, T. Linzen, G. Chrupała, Y. Belinkov, and D. Hupkes, Eds. Florence, Italy: Association for Computational Linguistics, Aug. 2019, pp. 276–286. [Online]. Available: https://aclanthology.org/W19-4828/

[12] O. Kovaleva, A. Romanov, A. Rogers, and A. Rumshisky, "Revealing the dark secrets of BERT," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4365–4374. [Online]. Available: https://aclanthology.org/D19-1445/

[13] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, "Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum, and L. Màrquez, Eds. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 5797–5808. [Online]. Available: https://aclanthology.org/P19-1580/

[14] P. Michel, O. Levy, and G. Neubig, "Are sixteen heads really better than one?" in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/2c601ad9d2ff9bc8b282670cdd54f69f-Paper.pdf

[15] I. Tenney, D. Das, and E. Pavlick, "BERT rediscovers the classical NLP pipeline," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum, and L. Màrquez, Eds. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 4593–4601. [Online]. Available: https://aclanthology.org/P19-1452/

[16] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," International Conference on Learning Representations, 2024, 2024. [Online]. Available: https://arxiv.org/abs/2309.17453

[17] S. Abnar and W. Zuidema, "Quantifying attention flow in transformers," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 4190–4197. [Online]. Available: https://aclanthology.org/2020.acl-main.385/

[18] S. Jain and B. C. Wallace, "Attention is not Explanation," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 3543–3556. [Online]. Available: https://aclanthology.org/N19-1357/

[19] S. Wiegreffe and Y. Pinter, "Attention is not not explanation," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 11–20. [Online]. Available: https://aclanthology.org/D19-1002/

[20] S. Serrano and N. A. Smith, "Is attention interpretable?" in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum, and L. Màrquez, Eds. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 2931–2951. [Online]. Available: https://aclanthology.org/P19-1282/

[21] G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui, "Attention is not only a weight: Analyzing transformers with vector norms," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, B. Webber, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 7057–7075. [Online]. Available: https://aclanthology.org/2020.emnlp-main.574/

[22] J. Bastings and K. Filippova, "The elephant in the interpretability room: Why use attention as explanation when we have saliency methods?" in *Proceedings of the Third BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, A. Alishahi, Y. Belinkov, G. Chrupała, D. Hupkes, Y. Pinter, and H. Sajjad, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 149–155. [Online]. Available: https://aclanthology.org/2020.blackboxnlp-1.14/

[23] S. Ma, H. Wang, S. Huang, W. Wang, Z. Chi, L. Dong, A. Benhaim, B. Patra, V. Chaudhary, X. Song, and F. Wei, "TorchScale: Transformers at Scale," arXiv:2211.13184, 2022. [Online]. Available: https://arxiv.org/abs/2211.13184

[24] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, H. D. III and A. Singh, Eds., vol. 119. PMLR, 13–18 Jul 2020, pp. 5156–5165. [Online]. Available: https://proceedings.mlr.press/v119/katharopoulos20a.html

[25] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with

Selective State Spaces," arXiv:2312.00752, 2023. [Online]. Available: https://arxiv.org/abs/2312.00752

[26] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, N. DasSarma, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah, "A Mathematical Framework for Transformer Circuits," Transformer Circuits Thread, 2021. [Online]. Available: https://transformer-circuits.pub/2021/framework/index.html

[27] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, S. Johnston, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah, "In-context Learning and Induction Heads," arXiv:2209.11895, 2022. [Online]. Available: https://arxiv.org/abs/2209.11895

[28] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Q. Liu and D. Schlangen, Eds. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. [Online]. Available: https://aclanthology.org/2020.emnlp-demos.6/

[29] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "The Pile: An 800GB Dataset of Diverse Text for Language Modeling," arXiv:2101.00027, 2020. [Online]. Available: https://arxiv.org/abs/2101.00027

[30] R. Eldan and Y. Li, "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?" arXiv:2305.07759, 2023. [Online]. Available: https://arxiv.org/abs/2305.07759

[31] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[32] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. [Online]. Available: https://matplotlib.org/stable/users/project/citing.html